

Ejercicio 7 — Casos de prueba funcionales (Programa COBOL)

Alcance

Sistema batch en COBOL que procesa los archivos `palabras.dat` e `idiomas.dat`, recibe una letra como parámetro y genera un reporte por idioma.

Las validaciones son **estrictas** y el programa **finaliza ante el primer error**.

Casos de prueba

TC-01 — Ejecución exitosa básica

Objetivo

Validar ejecución correcta con archivos válidos y parámetro válido.

Precondiciones

- Existen `palabras.dat` e `idiomas.dat`.
- Todos los registros cumplen el formato.
- Idiomas referenciados existen.

Pasos

1. Ejecutar el programa con parámetro `A`.
2. Procesar ambos archivos completos.

Resultado esperado

- El programa finaliza exitosamente.
- Se muestra el reporte por idioma.
- Se informan:
 - total de palabras
 - palabras que comienzan con `A`
 - promedio de longitud

TC-02 — Parámetro en minúscula

Objetivo

Validar normalización del parámetro de entrada.

Precondiciones

- Archivos válidos.
- Palabras que comienzan con **A**.

Pasos

1. Ejecutar el programa con parámetro **a**.

Resultado esperado

- El parámetro se normaliza a **A**.
- El resultado es idéntico a ejecutar con **A**.

TC-03 — Archivo palabras.dat inexistente

Objetivo

Validar control de existencia de archivos.

Precondiciones

- **idiomas.dat** existe.
- **palabras.dat** no existe.

Pasos

1. Ejecutar el programa.

Resultado esperado

- Se muestra mensaje de error indicando archivo inexistente.
- El programa finaliza inmediatamente.
- No se procesa ningún registro.

TC-04 — Registro con separadores incorrectos

Objetivo

Validar cantidad exacta de separadores.

Precondiciones

- Registro en `palabras.dat` con menos o más de dos `|`.

Pasos

1. Ejecutar el programa.

Resultado esperado

- Se detecta el error en el registro.
- Se informa el error y la línea.
- El programa finaliza inmediatamente.

TC-05 — Código no numérico

Objetivo

Validar formato del código.

Precondiciones

- Registro con código no numérico o con caracteres alfabéticos.

Pasos

1. Ejecutar el programa.

Resultado esperado

- Error informado: código inválido.
- El programa finaliza sin continuar.

TC-06 — Código numérico con longitud distinta de 5

Objetivo

Validar longitud exacta del código.

Precondiciones

- Código numérico con menos o más de 5 dígitos.

Pasos

1. Ejecutar el programa.

Resultado esperado

- Registro marcado como inválido.
- Mensaje de error mostrado.
- Finalización inmediata del programa.

TC-07 — Palabra con caracteres no alfabéticos

Objetivo

Validar restricción sobre el campo palabra.

Precondiciones

- Registro con palabra que contiene números o símbolos.

Pasos

1. Ejecutar el programa.

Resultado esperado

- Error por caracteres no alfabéticos.
- El programa finaliza inmediatamente.

TC-08 — Idioma inexistente

Objetivo

Validar referencia a idiomas cargados en memoria.

Precondiciones

- `palabras.dat` contiene un idioma no presente en `idiomas.dat`.

Pasos

1. Ejecutar el programa.

Resultado esperado

- Se informa idioma inexistente.
- El programa finaliza sin continuar.

TC-09 — Error luego de registros válidos

Objetivo

Validar corte ante primer error fatal.

Precondiciones

- Primeros registros válidos.
- Un registro inválido en una línea posterior.

Pasos

1. Ejecutar el programa.

Resultado esperado

- Los registros previos no generan salida parcial.
- Al detectar el error:
 - se informa
 - el programa finaliza
- No se continúa procesando registros siguientes.

TC-10 — Orden de archivos no garantizado

Objetivo

Validar independencia del orden de los archivos.

Precondiciones

- Ambos archivos válidos.
- El sistema operativo entrega los archivos en cualquier orden.

Pasos

1. Ejecutar el programa.

Resultado esperado

- `idiomas.dat` se carga completamente en memoria.
- `palabras.dat` se procesa correctamente.
- El resultado es consistente.

Cobertura y exclusiones

Cobertura lograda

- Validaciones de archivos
- Validaciones de formato
- Validaciones de dominio
- Normalización de parámetro
- Corte ante error
- Procesamiento correcto

Qué no se prueba (justificado)

- Rendimiento con volúmenes masivos (fuera del alcance funcional).
- Concurrencia (programa batch secuencial).
- Codificaciones regionales (no especificadas).
- Redondeo alternativo del promedio (no definido en consigna).

Decisiones asumidas ante aspectos no definidos en la consigna

Codificación de archivos de entrada

La consigna no define explícitamente la codificación de los archivos `palabras.dat` e `idiomas.dat`.

Se asume que los archivos se encuentran en codificación ASCII/UTF-8 sin caracteres especiales.

La presencia de caracteres fuera de este rango se considera inválida y provoca un error fatal, en línea con el criterio de validación estricta solicitado.

Esta decisión justifica el rechazo de palabras con caracteres acentuados o especiales.

Sensibilidad a mayúsculas y minúsculas en el idioma

No se especifica si el código de idioma debe tratarse de forma case-insensitive.

Se adopta una validación **case-sensitive**, comparando el idioma exactamente contra los valores cargados desde `idiomas.dat`.

No se realiza normalización automática, ya que asumirla implicaría introducir una regla no definida en la consigna.

Tratamiento de espacios en blanco

La consigna no define el tratamiento de espacios en blanco al inicio o fin de los campos.

No se realiza trimming automático de los valores; los espacios forman parte del dato leído y pueden invalidar el registro si afectan las validaciones.

Esta decisión prioriza la detección temprana de inconsistencias en los datos de entrada.

Formato del reporte de salida

El formato exacto del reporte no está definido.

El reporte se muestra por salida estándar, agrupado por idioma, informando claramente:

- cantidad total de palabras,
- cantidad de palabras que comienzan con la letra indicada,
- promedio de longitud de palabras.

No se genera salida parcial ante errores, respetando el criterio de corte inmediato.

Mejoras y extensiones no implementadas (fuera de alcance)

Contenido

Modo de validación sin corte

Como posible mejora futura, podría incorporarse un modo opcional que permita continuar el procesamiento ante errores, registrando todas las inconsistencias detectadas.

Este modo sería útil para análisis de calidad de datos, pero **no se implementa** porque la consigna exige finalizar ante el primer error fatal.

Logging estructurado

La consigna no define un esquema de logging.

En un entorno productivo, sería recomendable incorporar logs estructurados con niveles (INFO / ERROR), timestamps y redirección a archivo para facilitar auditoría y soporte operativo.

Métricas operativas

No se solicitan métricas de ejecución.

Como mejora, podrían incorporarse métricas como cantidad total de registros procesados, tiempo de ejecución y tasa de errores para mejorar la observabilidad del batch.

Validaciones adicionales de consistencia

No se define la necesidad de validaciones cruzadas adicionales entre archivos.

Podrían agregarse controles como detección de códigos duplicados o palabras repetidas por idioma como validaciones de calidad de datos, sin afectar la lógica principal.

Ejercicio 8 — Definición de comportamiento para casos ambiguos (Programa COBOL)

Sistema: programa batch COBOL que procesa `palabras.dat` e `idiomas.dat` con validaciones estrictas y **finaliza ante el primer error**.

Caso 1 — Idioma = `es`

Ambigüedad

No se define si el idioma es case-sensitive ni si se permiten códigos en minúscula.

Decisión

- El idioma se compara **case-sensitive** contra los valores cargados desde `idiomas.dat`.

Justificación

- COBOL no normaliza strings implícitamente.
- La consigna no indica normalización de idioma (a diferencia del parámetro).
- Normalizar sin indicación sería una suposición funcional.

Resultado esperado

- Si `idiomas.dat` contiene `es` exactamente → **válido**.
- Si contiene `ES` y el registro trae `es` → **idioma inexistente** → **error fatal**.

Caso 2 — Palabra `Árbol`

Ambigüedad

Contiene carácter alfabético acentuado (Unicode).

Decisión

- Se considera **inválida**.

Justificación

- La regla exige “caracteres alfabéticos” sin aclarar Unicode.
- En COBOL batch estándar, el chequeo alfabético suele ser ASCII (`A–Z`).

- Aceptar acentos introduce dependencias de encoding no definidas.

Resultado esperado

- Registro inválido.
- Mensaje de error indicando caracteres no alfabéticos.
- Finalización inmediata del programa.

Caso 3 — Línea vacía

Ambigüedad

No se especifica cómo tratar líneas vacías en archivos secuenciales.

Decisión

- Se considera **registro inválido**.

Justificación

- No contiene exactamente dos separadores.
- Tratarla como “ignorable” contradice validación estricta.
- Permitir líneas vacías introduce tolerancia no pedida.

Resultado esperado

- Error por formato inválido.
- Indicación de línea procesada.
- Finalización inmediata.

Caso 4 — Código 00000

Ambigüedad

Es numérico y tiene 5 dígitos, pero puede interpretarse como inválido semánticamente.

Decisión

- Se considera **válido**.

Justificación

- Cumple estrictamente las reglas: numérico y longitud 5.
- No se define rango mínimo ni exclusiones semánticas.
- Invalidarlo sería una regla inventada.

Resultado esperado

- El registro se procesa normalmente.
- Se incluye en conteos y cálculos.

Resumen de decisiones

Caso	Decisión	Motivo principal
Idioma es	Case-sensitiv e	No se define normalización
Árbol	Inválida	Alfabético ASCII implícito
Línea vacía	Inválida	Validación estricta
00000	Válida	Cumple reglas formales

Estas decisiones mantienen **consistencia con la validación estricta**, evitan suposiciones no pedidas.

Ejercicio 9 — Análisis de fallas (Programa COBOL)

Supuesto:

Todos los casos de prueba definidos en el Ejercicio 7 **fallan**.

Para cada caso se indica **severidad**, **impacto en el negocio** y **prioridad de automatización**.

Escala usada:

- **Severidad:** Baja / Media / Alta / Crítica
- **Prioridad de automatización:** Baja / Media / Alta

TC-01 — Ejecución exitosa básica

Falla asumida

El programa no genera el reporte correcto con datos válidos.

- **Severidad:** Crítica
- **Impacto en negocio:**
El sistema no cumple su función principal. No hay salida confiable aun con datos correctos.
- **Prioridad de automatización:** Alta
Caso base. Si falla, todo el sistema es inutilizable.

TC-02 — Parámetro en minúscula

Falla asumida

El programa no normaliza el parámetro y produce resultados incorrectos.

- **Severidad:** Alta
- **Impacto en negocio:**
Ejecuciones válidas fallan por un detalle de entrada menor, generando reprocesos manuales.
- **Prioridad de automatización:** Alta
Error frecuente y fácil de automatizar.

TC-03 — Archivo palabras.dat inexistente

Falla asumida

El programa no detecta la ausencia del archivo o falla de forma no controlada.

- **Severidad:** Crítica
- **Impacto en negocio:**
Caídas inesperadas del batch, posibles procesos posteriores inconsistentes.
- **Prioridad de automatización:** Alta
Validación básica de entorno productivo.

TC-04 — Registro con separadores incorrectos

Falla asumida

El programa continúa procesando registros mal formateados.

- **Severidad:** Alta
- **Impacto en negocio:**
Datos corruptos en reportes; pérdida de confiabilidad del resultado.
- **Prioridad de automatización:** Alta
Regla estricta y recurrente.

TC-05 — Código no numérico

Falla asumida

El código inválido no es detectado.

- **Severidad:** Alta
- **Impacto en negocio:**
Registros inconsistentes contaminan métricas y reportes.
- **Prioridad de automatización:** Media
Importante, pero menos frecuente que errores de formato.

TC-06 — Código con longitud incorrecta

Falla asumida

Se aceptan códigos fuera de especificación.

- **Severidad:** Media
- **Impacto en negocio:**
Inconsistencias de identificación; afecta trazabilidad, no el proceso completo.
- **Prioridad de automatización:** Media

TC-07 — Palabra con caracteres no alfabéticos

Falla asumida

El programa procesa palabras inválidas.

- **Severidad:** Media
- **Impacto en negocio:**
Métricas incorrectas (conteos y promedios), impacto analítico.
- **Prioridad de automatización:** Media

TC-08 — Idioma inexistente

Falla asumida

El programa no valida la existencia del idioma.

- **Severidad:** Alta
- **Impacto en negocio:**
Reportes por idioma incorrectos o incompletos.
- **Prioridad de automatización:** Alta

TC-09 — Error luego de registros válidos

Falla asumida

El programa no corta ante el primer error.

- **Severidad:** Crítica
- **Impacto en negocio:**
Procesamiento parcial silencioso; resultados imposibles de auditar.
- **Prioridad de automatización:** Alta

TC-10 — Orden de archivos no garantizado

Falla asumida

El programa depende del orden de lectura de archivos.

- **Severidad:** Alta
- **Impacto en negocio:**
Fallas intermitentes según entorno o scheduler; difícil diagnóstico.
- **Prioridad de automatización:** Alta

Resumen consolidado

Caso	Severidad	Impacto negocio	Prioridad automatización
TC-01	Crítica	Función principal inutilizable	Alta
TC-02	Alta	Reprocesos frecuentes	Alta
TC-03	Crítica	Caídas batch	Alta
TC-04	Alta	Datos corruptos	Alta
TC-05	Alta	Inconsistencia de datos	Media
TC-06	Media	Problemas de trazabilidad	Media
TC-07	Media	Métricas incorrectas	Media
TC-08	Alta	Reportes inválidos	Alta
TC-09	Crítica	Procesamiento parcial	Alta
TC-10	Alta	Fallas intermitentes	Alta

Este análisis prioriza **riesgo operativo y confiabilidad del batch**, alineado con un entorno productivo real.

Ejercicio 10 — Modelado de casos en Gherkin (Programa COBOL)

Sistema: programa batch COBOL que procesa `palabras.dat` e `idiomas.dat` y recibe una letra como parámetro.

Regla aplicada: un **Scenario por regla**.

Se reutiliza **el mismo archivo con distinto parámetro** en dos escenarios.

Feature: Procesamiento batch de palabras por idioma

El sistema procesa archivos secuenciales aplicando validaciones estrictas y genera un reporte por idioma.

Ante el primer error, el procesamiento se detiene inmediatamente.

Scenario 1 — Ejecución exitosa con datos válidos

Scenario: Procesar archivos válidos y generar reporte correctamente

Given existe el archivo "idiomas.dat" con idiomas válidos

And existe el archivo "palabras.dat" con registros bien formados

And el archivo "palabras.dat" referencia únicamente idiomas existentes

When ejecuto el programa con el parámetro "A"

Then el programa finaliza exitosamente

And se muestra un reporte por cada idioma

And se informa el total de palabras

And se informa la cantidad de palabras que comienzan con "A"

And se informa el promedio de longitud de palabras

Scenario 2 — Parámetro en minúscula (reutiliza mismos archivos)

Scenario: Normalizar parámetro en minúscula

Given existe el archivo "idiomas.dat" con idiomas válidos

And existe el archivo "palabras.dat" con registros bien formados

When ejecuto el programa con el parámetro "a"

Then el parámetro es normalizado a "A"

And el resultado es idéntico a ejecutar con el parámetro "A"

Scenario 3 — Archivo palabras.dat inexistente

Scenario: Fallar si el archivo palabras.dat no existe

Given existe el archivo "idiomas.dat"

And no existe el archivo "palabras.dat"

When ejecuto el programa con el parámetro "A"

Then se muestra un mensaje de error indicando archivo inexistente

And el programa finaliza inmediatamente

Scenario 4 — Registro con separadores incorrectos

Scenario: Detectar registro con formato inválido

Given existe el archivo "idiomas.dat"

And existe el archivo "palabras.dat" con un registro que no tiene exactamente dos separadores

When ejecuto el programa con el parámetro "A"

Then se muestra un mensaje de error de formato inválido

And se indica la línea procesada

And el programa finaliza inmediatamente

Scenario 5 — Código no numérico o con longitud incorrecta

Scenario: Detectar código inválido

Given existe el archivo "idiomas.dat"

And existe el archivo "palabras.dat" con un registro cuyo código no es numérico o no tiene cinco dígitos

When ejecuto el programa con el parámetro "A"

Then se muestra un mensaje de error por código inválido

And el programa finaliza inmediatamente

Scenario 6 — Palabra con caracteres no alfabéticos

Scenario: Detectar palabra con caracteres no alfabéticos

Given existe el archivo "idiomas.dat"

And existe el archivo "palabras.dat" con una palabra que contiene caracteres no alfabéticos

When ejecuto el programa con el parámetro "A"

Then se muestra un mensaje de error indicando palabra inválida

And el programa finaliza inmediatamente

Scenario 7 — Idioma inexistente

Scenario: Detectar idioma inexistente

Given existe el archivo "idiomas.dat"

And existe el archivo "palabras.dat" con un idioma no definido en idiomas.dat

When ejecuto el programa con el parámetro "A"

Then se muestra un mensaje de error indicando idioma inexistente

And el programa finaliza inmediatamente

Scenario 8 — Corte ante primer error

Scenario: Detener procesamiento ante el primer error

Given existe el archivo "idiomas.dat"

And existe el archivo "palabras.dat" con registros válidos seguidos de un registro inválido

When ejecuto el programa con el parámetro "A"

Then se procesan los registros válidos previos

And al detectar el error se muestra el mensaje correspondiente

And no se procesan registros posteriores

Scenario 9 — Orden de archivos no garantizado

Scenario: Procesar correctamente sin asumir orden de archivos

Given existen los archivos "idiomas.dat" y "palabras.dat"

And el sistema no garantiza el orden de lectura de los archivos

When ejecuto el programa con el parámetro "A"

Then el archivo idiomas.dat se carga completamente en memoria

And el archivo palabras.dat se procesa correctamente

And el resultado es consistente

Observaciones

- Se cumple **un Scenario por regla**.
- Los escenarios 1 y 2 **reutilizan los mismos archivos con distinto parámetro**, como exige la consigna.
- Los **Given** son realistas y coherentes con un entorno batch.
- No se asumen comportamientos no definidos.

Este modelado es directamente utilizable para el Ejercicio 11 (Cucumber).

Ejercicio 12- Reflexión técnica

Qué fue lo más difícil de testear

Lo más complejo de testear fue el comportamiento del sistema ante errores fatales. El programa está diseñado para finalizar inmediatamente al detectar la primera condición inválida, lo que obliga a construir casos de prueba muy controlados donde el error esperado ocurra exactamente en la línea prevista. Esto limita la observación de estados intermedios y dificulta validar escenarios más amplios dentro de una misma ejecución.

Además, el uso de archivos secuenciales introduce una fuerte dependencia con el entorno de ejecución (existencia de archivos, permisos, formato y encoding), lo que hace más difícil aislar la lógica del programa del contexto operativo real.

Qué riesgos quedan sin cubrir

Quedan sin cubrir riesgos relacionados con rendimiento y escalabilidad, como el procesamiento de archivos de gran volumen o tiempos de ejecución elevados, ya que no forman parte del alcance funcional.

Tampoco se abordan riesgos asociados a diferentes codificaciones de caracteres o configuraciones regionales, dado que el comportamiento esperado no está explícitamente definido y se asume un entorno controlado.

Por último, no se contemplan escenarios de concurrencia ni ejecuciones paralelas, ya que el sistema corresponde a un proceso batch secuencial.

Qué automatizarías primero en un entorno bancario real

En un entorno bancario real, priorizaría la automatización de las validaciones críticas que impactan directamente en la integridad de los datos y la confiabilidad del proceso, especialmente los controles de formato, reglas de negocio estrictas y manejo de entradas inválidas.

También automatizaría tempranamente los escenarios de error fatal y corte controlado del proceso, ya que una falla mal gestionada en un batch bancario puede generar reprocesos costosos, demoras operativas o inconsistencias contables.

Finalmente, daría alta prioridad a la automatización de pruebas de regresión sobre procesos batch críticos, integrándolas al pipeline de despliegue para detectar errores antes de su ejecución en producción.

Propuesta de evolución del sistema

Aunque la prueba no requiere un sistema productivo completo, la solución podría evolucionar hacia un pipeline batch más robusto incorporando:

- validaciones configurables,
- métricas de ejecución,
- logging estructurado,
- integración con schedulers corporativos,
- pruebas automatizadas integradas al pipeline de despliegue.

Estas extensiones no se implementan para mantener el foco en los requerimientos explícitos, pero demuestran cómo la solución podría escalar en un entorno real.